theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)
For a small imperative language `StImp` with mutable state, the entailment $[t,\ m \Vdash_L\ \varphi]_S$ is proved structurally via inference rules on the syntax of t.

Key rules from `imp-lemmas.tex`:

- **Seq**$_S$AU$_L$: sequential composition decomposes into bind
- **If**$_S$AU$_L$: conditional branches both satisfy the obligation
- **While**$_S$AG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**$_S$AU$_L$: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define `wp(_, _)` for Vera.

Notation

$$\sigma \in \text{State (mutable references, heap)}$$

$$[t,\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}} \triangleq \text{“Rust term } t \text{ from state } \sigma \text{ satisfies } \varphi\text{”}$$

$$\texttt{wp}(t,\ \varphi) \triangleq \text{weakest precondition on } \sigma \text{ s.t. } [t,\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}}$$

Temporal formulas

- $\varphi ::= \texttt{now}\ P \mid \texttt{AG}\ \varphi \mid \varphi\ \texttt{AU}\ \varphi' \mid \varphi \wedge \varphi'$
- Sugar: $\texttt{AF}\ \varphi' = \top\ \texttt{AU}\ \varphi', \quad \texttt{AX}\ \varphi' = \top\ \texttt{AN}\ \varphi'$
- $\texttt{done}\ Q$: termination postcondition; $\texttt{now}\ P$: intermediate state predicate

Inference Rules

Assignment

Rust: $*x = e$

$$\frac{[e,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}\ \mathcal{R}]_{\texttt{Rust}} \qquad \forall\ v, \sigma',\ \mathcal{R}\ v\ \sigma' \to [\texttt{skip},\ \sigma'[x \mapsto v] \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}{[*x = e,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{Assign}_{\textsc{Rust}}\textsc{AU}_L$$

Evaluating e may take steps (function calls, etc.); φ must hold along the path. After e produces v, the store $\sigma'[x \mapsto v]$ must satisfy φ'.

Sequential Composition (let-binding)

Rust: $\texttt{let}\ x = t;\ u$

$$\frac{[t,\ \sigma \Vdash_R\ \ \varphi\ \texttt{AU}\ \texttt{AX}\ \texttt{done}\ \mathcal{R}]_{\texttt{Rust}} \qquad \forall\ x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [u[x],\ \sigma' \Vdash_L\ \ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}{[\texttt{let}\ x = t;\ u,\ \sigma \Vdash_L\ \ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{Let}_{\textsc{Rust}}\textsc{AU}_L$$

Direct lift of **BindAU**$_L$ / **Seq**$_S$AU$_L$ from TICL.

Conditional

Rust: $\texttt{if}\ c\ \{t\}\ \texttt{else}\ \{u\}$

$$\frac{\begin{array}{l}[c,\ \sigma \Vdash_R\ \varphi\ \texttt{AU}\ \texttt{done}_{=}\ (b, \sigma')]_{\texttt{Rust}} \\ \begin{cases}[t,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}, & \text{if } b \\ [u,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}, & \text{otherwise}\end{cases}\end{array}}{[\texttt{if}\ c\ \{t\}\ \texttt{else}\ \{u\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{If}_{\textsc{Rust}}\textsc{AU}_L$$

Evaluating c may take multiple steps; φ must hold along the path.

While Loop (AU – terminating)

Rust: `while` c $\{t\}$ **with** `invariant` $\mathcal{R}$, `decreases` f

$$\frac{\forall\ \sigma,\ \mathcal{R}\ \sigma \to [c,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}_{=}\ (b, \sigma')]_{\texttt{Rust}}^{\mathcal{R}\ \sigma} \qquad \wedge \begin{cases} [t,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}} \vee \\ \quad [t,\ \sigma' \Vdash_R\ \varphi\ \texttt{AU AF done}\ (\lambda\ \sigma''.\ \mathcal{R}\ \sigma'' \wedge f\ \sigma'' < f\ \sigma)]_{\texttt{Rust}}, & \text{if } b \\ \varphi'(\sigma'), & \text{otherwise} \end{cases}}{[\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{While}_{\textsc{Rust}}\textsc{AU}_L$$

Loop (AG – infinite)

Rust: $\texttt{loop}\ \{t\}$ **with** $\texttt{invariant}\ \mathcal{R}$ **(no** $\texttt{decreases}$**)**

$$\frac{\begin{array}{l}\mathcal{R}\ \sigma \\ \forall\ \sigma,\ \mathcal{R}\ \sigma \rightarrow \\ \quad [\texttt{loop}\ \{t\},\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}} \wedge \\ \quad [t,\ \sigma \Vdash_R\ \varphi\ \texttt{AU}\ \ \texttt{AF done}\ (\lambda\ \sigma' \Rightarrow \mathcal{R}\ \sigma')]_{\texttt{Rust}}\end{array}}{[\texttt{loop}\ \{t\},\ \sigma \Vdash_L\ \ \texttt{AG}\ \varphi]_{\texttt{Rust}}}\ \textsc{Loop}_{\textsc{Rust}}\textsc{AG}$$

Coinductive hypothesis $[\texttt{loop}\ \{t\},\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}}$ enables proving φ holds at loop re-entry after each iteration.

Async Bind (Lazy Future Creation)

Rust: $\texttt{let}\ p = \texttt{async}\ f(\vec{e});\ k\ p$

$$\frac{\begin{array}{c}\text{No state change (lazy future)}\\ [k\ p,\ \sigma \Vdash_{\mathcal{L}}\ \Phi]_{\texttt{Rust}}\end{array}}{[\texttt{let}\ p = \texttt{async}\ f(\vec{e});\ k\ p,\ \sigma \Vdash_{\mathcal{L}}\ \Phi]_{\texttt{Rust}}}\ \textsc{AsyncBind}_{\textsc{Rust}}$$

Rust futures are lazy: `async fn()` returns a handle, body runs at `.await`.

Await (AU/AF – terminating callee)

Rust: $\texttt{let } x = p.\texttt{await}; \; k \; x$

$$\frac{\begin{array}{c}[P[p],\ \sigma \Vdash_R\ \varphi'\ \texttt{AU AX done}\ \mathcal{R}]_{\texttt{Rust}} \\ \varphi' \implies \varphi \quad \text{(transparent)} \\ \forall\ x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [k\ x,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'']_{\texttt{Rust}}\end{array}}{[\texttt{let } x = p.\texttt{await}; \; k \; x,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'']_{\texttt{Rust}}}\ \textsc{Await}_{\textsc{Rust}}\textsc{AU}_L$$

$P[p]$: async process behind future p. Callee's path φ' must imply caller's φ (transparent await).

Await (AG – diverging callee)

Rust: $p.\texttt{await}$ **where** $P[p]$ **satisfies** $\texttt{AG}\ \psi$

$$\frac{[P[p],\ \sigma \Vdash_L\ \texttt{AG}\ \psi]_{\texttt{Rust}} \qquad \psi \implies \varphi}{[p.\texttt{await},\ \sigma \Vdash_L\ \texttt{AG}\ \varphi]_{\texttt{Rust}}}\ \textsc{Await}_{\textsc{Rust}}\textsc{AG}$$

Callee runs forever $\rightarrow$ await is a *divergence point*. Caller's $\texttt{AG}\ \varphi$ discharged if $\psi \implies \varphi$.

Weakest Preconditions

wp: Assignment

$$\begin{aligned}\texttt{wp}(*x = e,\ \varphi\ \texttt{AU}\ \varphi') \ &= \ \lambda\,\sigma.\ \ \texttt{wp}(e,\ \varphi\ \texttt{AU done}\ \mathcal{R})\ \sigma \\ &\qquad \wedge\ \ \forall\ v, \sigma'.\ \mathcal{R}\ v\ \sigma' \to \varphi'(\sigma'[x \mapsto v])\end{aligned}$$

Evaluate e (maintaining φ), then check φ' after the store.

wp: Let-binding (Sequential Composition)

$$\begin{aligned}\texttt{wp}(\texttt{let}\ x = t;\ u,\ \varphi\ \texttt{AU}\ \varphi') \ =\ & \lambda\,\sigma.\ \ \texttt{wp}(t,\ \varphi\ \texttt{AU}\ \texttt{AX}\ \texttt{done}\ \mathcal{R})\ \sigma \\ & \wedge\ \ \forall x, \sigma'.\ \mathcal{R}\ x\ \sigma' \to \texttt{wp}(u[x],\ \varphi\ \texttt{AU}\ \varphi')\ \sigma'\end{aligned}$$

wp: Conditional

$$\mathtt{wp}(\mathtt{if}\ c\ \{t\}\ \mathtt{else}\ \{u\},\ \varphi\ \mathtt{AU}\ \varphi') \;=\; \lambda\,\sigma.\ \ \mathtt{wp}(c,\ \varphi\ \mathtt{AU}\ \mathtt{done}_{=}\ (b, \sigma'))\ \sigma \;\wedge\; \begin{cases} \mathtt{wp}(t,\ \varphi\ \mathtt{AU}\ \varphi')\ \sigma', & \text{if } b \\ \mathtt{wp}(u,\ \varphi\ \mathtt{AU}\ \varphi')\ \sigma', & \text{otherwise} \end{cases}$$

wp: While Loop (AU)

$$
\begin{aligned}
&\mathtt{wp}(\mathtt{while}\ c\ \{t\},\ \varphi\ \mathtt{AU}\ \varphi')\ =\ \lambda\,\sigma.\\
&\quad \mathcal{R}\ \sigma\ \wedge\ \forall\,\sigma,\ \mathcal{R}\ \sigma \rightarrow\\
&\quad \mathtt{wp}(c,\ \varphi\ \mathtt{AU}\ \mathtt{done}_{=}\ (b, \sigma'))\ \sigma\ \wedge\\
&\quad \begin{cases}
\mathtt{wp}(t,\ \varphi\ \mathtt{AU}\ \varphi')\ \sigma'\ \vee & \\
\quad \mathtt{wp}(t,\ \varphi\ \mathtt{AU}\ \mathtt{AF}\ \mathtt{done}\ (\lambda\,\sigma''.\ \mathcal{R}\ \sigma'' \wedge f\ \sigma'' < f\ \sigma)) & \\
\quad\quad \sigma', & \text{if } b\\
\varphi'(\sigma'), & \text{o.w.}
\end{cases}
\end{aligned}
$$

wp: Loop (AG)

$$
\begin{aligned}
&\texttt{wp}(\texttt{loop}\ \{t\},\ \texttt{AG}\ \varphi)\ =\ \lambda\,\sigma.\\
&\quad \mathcal{R}\ \sigma\ \wedge\ \forall\,\sigma,\ \mathcal{R}\ \sigma \to\\
&\quad [\texttt{loop}\ \{t\},\ \sigma \Vdash_{\mathcal{L}}\ \varphi]_{\texttt{Rust}}\ \wedge\\
&\quad \texttt{wp}(t,\ \varphi\ \texttt{AU}\ \ \texttt{AF done}\ (\lambda\,\sigma'.\ \mathcal{R}\ \sigma'))\ \sigma
\end{aligned}
$$

Coinductive: $[\texttt{loop}\ \{t\},\ \sigma \Vdash_{\mathcal{L}}\ \varphi]_{\texttt{Rust}}$ is the coinductive hypothesis.

wp: Function Call – f ensures $\texttt{AF done}\ \mathcal{R}_f$

f has terminating postcondition

$$\texttt{wp}(f(e_1, \ldots, e_i),\ \varphi\ \texttt{AU}\ \varphi') \ =\ \lambda\, \sigma.\ \forall\, x, \sigma'.\ \mathcal{R}_f\ x\ \sigma' \to \texttt{wp}(k\ x,\ \varphi\ \texttt{AU}\ \varphi')\ \sigma'$$

where k is the continuation after the call.

f has temporal ensures Ψ: transparent checking
Additionally assert $\Psi \implies \Phi$ where Φ is the caller's obligation.

f ensures $\texttt{AG}\ \psi$: cannot prove $\texttt{AF}$ for it
If f has $\texttt{AG}\ \psi$ (diverges), then $\texttt{wp}(f(\vec{e}),\ \texttt{AF}\ \varphi')$ is **unprovable** – a diverging call can never reach the $\texttt{AF}$ goal.

wp: Async Bind

$$\texttt{wp}(\texttt{let } p = \texttt{async } f(\vec{e}); \; k \; p, \; \Phi) \; = \; \lambda \, \sigma. \; \texttt{wp}(k \; p, \; \Phi) \; \sigma$$

No-op: lazy future creation does not change state.

wp: Await (AU)

$$\texttt{wp}(\texttt{let } x = p.\texttt{await}; \; k \; x, \; \varphi \; \texttt{AU} \; \varphi'') \; = \; \lambda \, \sigma. \;\; \varphi' \implies \varphi$$
$$\wedge \;\; \forall \, x, \sigma'. \; \mathcal{R} \; x \; \sigma' \to \texttt{wp}(k \; x, \; \varphi \; \texttt{AU} \; \varphi'') \; \sigma'$$

where $P[p]$ $\texttt{ensures}$ φ' $\texttt{AU}$ $\texttt{AX}$ $\texttt{done}$ $\mathcal{R}$.

wp: Await (AG)

$$\texttt{wp}(p.\texttt{await},\ \texttt{AG}\ \varphi)\ =\ \lambda\,\sigma.\ \psi\implies\varphi$$

where $P[p]$ `ensures` AG ψ.

Callee diverges $\rightarrow$ continuation is dead code $\rightarrow$ no further wp.

Summary

Complete Rule Set

Rust Construct	Rule	Section
$*x = e$	State update, check φ	Inference
`let` $x = t$; u	BindAU_L	Inference
`if` c $\{t\}$ `else` $\{u\}$	Case split (effectful c)	Inference
`while` c $\{t\}$ + `decreases`	IterAU_L	Inference
`loop` $\{t\}$ (no exit)	IterAG (coinductive)	Inference
`let` $p =$ `async` $f(\vec{e})$; k	No-op (lazy)	Inference
p`.await` (AU callee)	Bind + transparent	Inference
p`.await` (AG callee)	Divergence	Inference
$f(\vec{e})$ (AF ensures)	Bind + assume $\mathcal{R}_f$	WP only
$f(\vec{e})$ (AG ensures)	Transparent $\Psi \implies \Phi$	WP only
$f(\vec{e})$ (AG + AF goal)	**Unprovable**	WP only

Function calls are contract-dependent $\rightarrow$ only in wp, not inference rules.